A Hardware Rendering of the Rho Calculus with Boolean Arithmetic

Design specification: source language, target machine, and verification guidance

F1R3FLY.io design note — draft 0.3

13 November 2026

Abstract

This note specifies a first hardware rendering of the rho calculus extended with Boolean arithmetic. It fixes the source language (syntax, sorts, structural congruence, matching modulo the Boolean laws, and reduction), the canonical forms that make names comparable in constant time, a synchronous tuple-space machine that realises the calculus on an FPGA and ports unchanged to a CMOS ASIC flow, the division of the implementation between Bluespec and Clash, and the obligations an implementation must discharge to count as verified. All decisions in the register of Section 2 are confirmed. One semantic question remains open and is stated in Section 10. Draft 0.3 folds in six corrections found by building and testing the reference implementation; they are listed in Section 11.

Contents

1 Purpose and scope **2**

2 Decision register **3**

3 Source language **3**
3.1 Syntax . . . 3
3.2 Sorts . . . 4
3.3 Structural congruence . . . 4
3.4 Matching . . . 4
3.5 Reduction . . . 5
3.6 Replication . . . 5

4 Canonical forms **5**

5 Target machine **6**
5.1 Overview . . . 6
5.2 Words and nodes . . . 6
5.3 Heap, interner and reference counts . . . 7
5.4 Channel store . . . 7
5.5 Matcher and mask solver . . . 8
5.6 Arbiter and entropy . . . 8
5.7 The step . . . 9
5.8 Instantiation . . . 10
5.9 Boundary . . . 10
5.10 Decision log and faults . . . 10
5.11 Parameters . . . 11

6 Tooling and code organisation **11**
6.1 Division of labour (**D9**) . . . 11
6.2 Rules for the Bluespec code . . . 11
6.3 Suggested module list . . . 12
6.4 Flow . . . 12

7 Compiler and image **12**

8 Verification **12**
8.1 What "verified" means . . . 12
8.2 Layers of evidence . . . 13
8.3 The step-refinement checker . . . 14
8.4 Invariants . . . 14
8.5 Directed test programs . . . 14

9 Staging **15**

10 Open question and recorded limits **15**

11 Status and changes in draft 0.3 **15**

A Worked examples **16**

1 Purpose and scope

The goal is a machine whose instruction set *is* the calculus: the only state change is the COMM rule, and every source of nondeterminism in the calculus is a physical arbitration in the machine. The first target is an FPGA, described in a high-level hardware language that emits Verilog. The design is restricted to components expressible in synthesizable Verilog or VHDL for CMOS; clockless and superconducting realisations are out of scope, but the interfaces are kept latency-insensitive so that a later self-timed version does not require a redesign.

In scope: the calculus of Section 3; a single-reducer machine with a path to parallel reduction; a compiler from source text to a memory image; a verification plan. Out of scope for this draft: cost accounting, join patterns, `where` clauses and grading, the combinator instruction set, and multi-chip sharding. Hooks are left where these attach (Sections 5.6 and 9).

2 Decision register

ID	Decision	Status
D1	Boolean operands admit drops: $*x$ is a Boolean expression.	C
D2	The Boolean-algebra laws are part of structural congruence.	C
D3	Replication is derivable by the reflective encoding; a persistent receive is provided as sugar and implemented as a primitive.	C
D4	Ill-sorted terms are excluded by a basic spatial-behavioural type system, not by the machine.	C
D5	Distinct solutions of a Boolean pattern are distinct, observable races, on the same footing as COMM races.	C
D6	First target is an FPGA, via generated Verilog.	C
D7	A high-level functional hardware language is used in place of hand-written RTL.	C
D8	Architecture: tuple-space machine (hash-consed term heap, channel store, matcher, arbiter). Static elaboration of the Boolean-typed fragment is a later optimisation.	C
D9	Tool split: Bluespec (BH syntax) for the rule-based reaction core; Clash for pure datapath functions; one Haskell reference library shared with the compiler.	C
D10	The compiler is trusted for sorts. The machine keeps a sticky `FAULT_SORT` flag whose unreachability is a proof obligation. The flag is a debugging aid: it does not detect every ill-sorted program.	C
D11	Variables are a separate syntactic category V, instantiated in the machine as de Bruijn indices. Names remain quoted processes.	C
D12	Open Boolean code is canonicalised as a truth table over at most $K = 6$ variables; larger supports are rejected at compile time.	C
D13	Boolean patterns (other than a bare variable) match closed Boolean values only.	C
D14	Fairness is possibilistic: every enabled redex has nonzero probability at every step. Uniformity over redexes is not required.	C
D15	Entropy is an LFSR behind an interface; every arbitration is written to a decision log that makes runs replayable.	C
D16	The machine is board-agnostic: no vendor primitives, inferred block RAM only, all sizes are parameters.	C

3 Source language

3.1 Syntax

Let V be a countable set of variables, ranged over by v, w. Processes P, Q, Boolean expressions B, names x, y, patterns p and deep patterns d are given by

$$
\begin{array}{rcl}
P, Q & ::= & \mathtt{0} \mid B \mid \mathtt{for}(p \leftarrow x)\, P \mid \mathtt{for}(p \Leftarrow x)\, P \mid x!(Q) \mid P \mid Q \mid *x \\
B & ::= & \mathtt{true} \mid \mathtt{false} \mid *x \mid \sim B \mid B \mathbin{\&\&} B \mid B \mathbin{||} B \mid B \mathbin{\mathtt{xor}} B \\
x, y & ::= & @P \mid v \\
p & ::= & v \mid @d \\
d & ::= & \mathtt{0} \mid \mathtt{true} \mid \mathtt{false} \mid v \mid p!(d) \mid \mathtt{for}(v \leftarrow p)\, d \mid \sim d \mid d \mathbin{\&\&} d \mid d \mathbin{||} d \mid d \mathbin{\mathtt{xor}} d
\end{array}
$$

Relative to the grammar that opened the session, **D1** adds $*x$ to B, **D3** adds the persistent receive $\mathtt{for}(p \Leftarrow x)\, P$, and **D11** separates variables from names. Quotation is written $@P$ and dereference $*x$ throughout. The production $*x$ is shared by P and B; which reading applies is settled by sorts.

A receive binds the variables of its pattern in its body. Every variable denotes a *name*. A variable in a deep-pattern position denotes the name of the matched subprocess, so matching $@(v \mathbin{\&\&} w)$ binds v and w to names of Booleans, and the body uses them as $*v$ and $*w$. A program is a closed process.

3.2 Sorts

There are two sorts of process, $\mathsf{Bool} \subseteq \mathsf{Proc}$, and correspondingly two sorts of variable: *Boolean-sorted* variables, which only ever denote $@\texttt{true}$ or $@\texttt{false}$, and *structural* variables, which may denote any name. A term is well sorted when every operand of a Boolean operator is of sort Bool; in particular $*v$ as an operand requires v to be Boolean-sorted, and a Boolean-sorted variable requires every message that can reach its receive to quote a Boolean.

The last condition is a property of channels, and is discharged by the spatial-behavioural type system (**D4**), which assigns a carried sort to each namespace. This note treats that system as a black box with the following interface.

- **Input.** A closed process.
- **Output.** Either a rejection, or a sort for every pattern variable, such that reduction preserves well-sortedness.

The reflective encoding of replication sends a quoted process on a channel that the process itself mentions, so the system needs a recursive formula to type it. Giving the persistent receive its own typing rule avoids that for programs that use only the sugar.

3.3 Structural congruence

Structural congruence $\equiv$ is the least congruence on processes containing:

1. alpha-equivalence of bound variables;
2. $(P \mid Q) \mid R \equiv P \mid (Q \mid R), \quad P \mid Q \equiv Q \mid P, \quad P \mid \mathtt{0} \equiv P$;
3. the laws of Boolean algebra on B, with $\texttt{xor}$ defined by $B_1 \mathbin{\texttt{xor}} B_2 \equiv (B_1 \mathbin{\&\&} {\sim}B_2) \mathbin{||} ({\sim}B_1 \mathbin{\&\&} B_2)$;
4. $*@P \equiv P$.

Name equivalence $\equiv_N$ is the least equivalence with $@P \equiv_N @Q$ whenever $P \equiv Q$, and $@{*x} \equiv_N x$. Law 4 is adopted as a convention; in the original presentation the same effect arises only through substitution.

By **D2** every closed Boolean expression is congruent to exactly one of $\texttt{true}$ or $\texttt{false}$, so there are exactly two closed Boolean names up to $\equiv_N$. The requirement that substituting a Boolean name into $*y$ yields the *evaluation* of the Boolean is therefore a consequence of law 3 and not a separate rule: evaluation is normalisation.

3.4 Matching

A substitution σ maps variables to closed names and respects sorts.

Definition 1 (Match set)**.** For a pattern p and a closed name n,

$$\mathsf{match}(p, n) \;=\; \{\, \sigma \mid \mathrm{dom}(\sigma) = \mathrm{vars}(p),\ p\sigma \equiv_N n \,\}/{\equiv_N}$$

where substitutions are identified when they agree pointwise up to $\equiv_N$.

Matching is *modulo* $\equiv$. For structural patterns this is ordinary tree matching on normal forms, and the match set has at most one element. For Boolean patterns it is equation solving: $\mathsf{match}(@(v \mathbin{\&\&} w), @\texttt{false})$ has three elements and $\mathsf{match}(@(v \mathbin{\&\&} w), @\texttt{true})$ has one. Because

Boolean-sorted variables range over two classes, a pattern with k Boolean variables has at most 2^k solutions.

Restrictions.

- (**D13**) A Boolean pattern built with an operator matches only a closed Boolean value. Matching an operator pattern against open Boolean code under a binder would be unification of Boolean functions and is excluded. A Boolean leaf, a constant leaf included, that meets a name not quoting a Boolean is a sort fault and not a failed match; so the pattern @`true` cannot be used to test whether a message is a Boolean.
- A pattern variable is a *mask variable* when it occurs under a Boolean operator in the pattern; every occurrence of it is then a Boolean leaf. Any other variable is bound structurally, whatever its sort, and its sort is checked where the body uses it. A pattern has at most K mask variables (**D12**).
- Deep patterns contain no parallel composition, as in the given grammar, so no matching modulo associativity and commutativity is needed.
- The receive pattern $\texttt{for}(v \leftarrow p)\, d$, which matches under a binder, is deferred; see Section 10.

3.5 Reduction

$$\frac{x \equiv_N y \qquad \sigma \in \mathsf{match}(p, @Q)}{\texttt{for}(p \leftarrow x)\, P \mid y!(Q) \;\rightarrow\; P\sigma} \;(\textsc{comm})$$

$$\frac{x \equiv_N y \qquad \sigma \in \mathsf{match}(p, @Q)}{\texttt{for}(p \Leftarrow x)\, P \mid y!(Q) \;\rightarrow\; P\sigma \mid \texttt{for}(p \Leftarrow x)\, P} \;(\textsc{comm}!)$$

$$\frac{P \rightarrow P'}{P \mid Q \rightarrow P' \mid Q} \;(\textsc{par}) \qquad \frac{P \equiv P' \quad P' \rightarrow Q' \quad Q' \equiv Q}{P \rightarrow Q} \;(\textsc{equiv})$$

There is no reduction under a receive or under a quote. Substitution $P\sigma$ replaces free variables by names and does not descend into closed quoted terms, which have no free variables.

A *redex* of a process is a triple (send, receive, solution). By **D5** two redexes that differ only in the solution are different transitions. A send whose channel has a receive but whose payload admits no solution is not a redex and stays in place.

3.6 Replication

With $D(x) = \texttt{for}(v \leftarrow x)\,(x!(*v) \mid *v)$, replication of P on a name x not otherwise used is $x!(D(x) \mid P) \mid D(x)$. The machine must run this term correctly without special support, which requires a general drop of quoted code. The persistent receive is the efficient form. Its specification is the encoding:

Obligation 1. For all p, x, P, the process $\texttt{for}(p \Leftarrow x)\, P$ is weakly bisimilar to its reflective encoding, under context-labelled transitions.

4 Canonical forms

The machine compares names by comparing machine words. That is sound only if every term is stored in a normal form that is unique up to $\equiv$. The normal form $\mathsf{nf}(P)$ is defined bottom-up.

Variables. Bound variables are replaced by de Bruijn indices: an occurrence is the number of variable binders between it and its own binder, counting each variable of a multi-variable pattern in left-to-right order of first occurrence. Closed subterms therefore have a representation that does not depend on their context, which is what permits sharing. This indexes V only; names are still quoted processes (**D11**).

Quote and drop. $*@P$ is replaced by $\mathsf{nf}(P)$, and $@*x$ by x.

Booleans. A Boolean expression whose operands are all closed is replaced by its value. Otherwise let $v_0 < \cdots < v_{k-1}$ be the variables it *depends on* (its support), ordered by index. The normal form is the pair of the support and the truth table $T \in \{0,1\}^{2^k}$, where bit $r = \sum_i b_i 2^i$ of T is the value of the expression at $v_i \mapsto b_i$. Variables on which the function does not depend are removed. A function that is the identity on a single variable v is stored as $*v$. If $k > K$ the program is rejected (**D12**).

Parallel composition. Compositions are flattened, `0` units are removed, and the components are sorted into ascending order of their machine words. Since each component is itself canonical and hash-consed (Section 5.3), word order is a total order on components up to $\equiv$. Duplicates are kept: composition is a multiset.

Other constructors. Sends, receives and persistent receives are normalised componentwise. Patterns are normalised by the compiler into pattern programs (Section 7); two receives are congruent only if their pattern programs are identical.

Obligation 2 (Canonicity)**.** For all well-sorted P, Q within the size bounds: $\mathsf{nf}(P) = \mathsf{nf}(Q)$ if and only if $P \equiv Q$.

Remark 1. The truth table over a minimal, ordered support is a canonical form for Boolean functions, so the Boolean part of Obligation 2 is immediate. The same 64-bit word serves three purposes: it is the normal form, it evaluates the function forwards (index it by the argument values), and it solves patterns backwards (Section 5.5).

5 Target machine

5.1 Overview

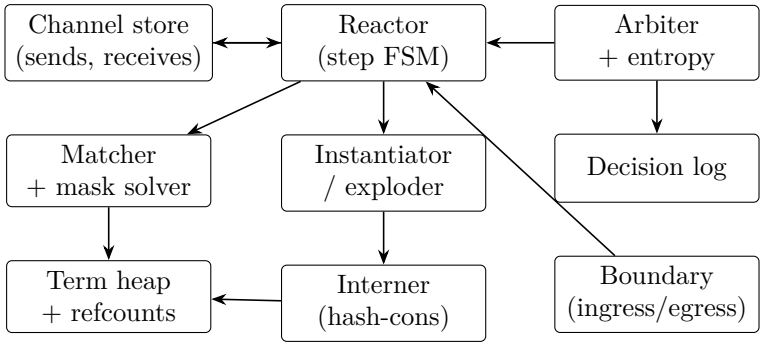


Figure 1: Blocks of the machine. Arrows show the direction of requests.

The machine holds the top level of the running process as a *soup*: a table of sends and a table of receives. Everything quoted lives in a hash-consed heap of canonical terms. One *step* selects a redex, removes its send (and its receive, unless persistent), and adds the components of the instantiated continuation to the soup. A step takes many clock cycles; the calculus sees only step boundaries.

5.2 Words and nodes

All widths are parameters; the reference values are for a 32-bit word with a 28-bit heap address.

Word	Tag	Meaning
Process word	`ZERO`	the process `0`
	`BOOL` b	the Boolean value b
	`PTR` a	the process stored at heap address a
Name word	`QZERO`	the name @`0`
	`QBOOL` b	the name @b
	`QPTR` a	the name @(node at a)
	`VAR` i	the variable with de Bruijn index i (code only)

Closed Boolean names are immediates and never touch the heap.

Node kind	Fields
`PAR`	head: process word; tail: process word (a `PAR` or the last component)
`SEND`	chan: name word; payload: process word
`RECV`	chan: name word; pat: pattern-program address; body: process word; nvars; persistent bit
`DROP`	name: name word (always a `VAR`, by normalisation)
`BFN`	arity $k \leq K$; support: k variable indices, ascending; table: 2^K bits

Every node carries a header with its kind, a *free-variable depth*, and a *sort* bit (the node is of sort Bool). The depth of a node is the least k such that every free variable index in it is below k; a node is *closed* when its depth is zero. Both fields are computed at interning time from the children: a variable i has depth $i + 1$; a receive with n pattern variables has depth $\max(d_{chan}, d_{body} - n, 0)$; every other node takes the maximum over its fields. A single closed bit would not do: whether a receive is closed depends on whether its body's free variables are all bound by its own pattern, which a bit on the body cannot tell. The exact packing into block-RAM words is left to the implementation; it must be fixed in one shared definition used by the compiler, the reference model and the hardware.

5.3 Heap, interner and reference counts

- **Heap.** An array of nodes in block RAM with a free list.
- **Interner.** `intern(node)` returns the address of the unique stored node equal to its argument, allocating if there is none. It is a multi-cycle operation: hash the node contents, probe a hash table in block RAM, and confirm any hit by a full field comparison. Equality is exact; the hash only selects the bucket, so there is no collision risk.
- **Normalising constructors.** The interner is the only way to create a node, and each constructor normalises: `PAR` performs the sorted insertion of Section 4; `BFN` minimises the support and collapses constants and projections; `DROP` of a closed name returns the named process.
- **Reference counts.** Terms are acyclic, so reference counting reclaims everything. A count is held per node in a separate RAM. Decrements that reach zero push the node on a deferred-free stack that is drained when the reactor is idle, which bounds the work per cycle.

5.4 Channel store

Two tables, each with a valid bit per entry.

- **Send table** $S[0..N_S)$: channel (closed name word), payload (closed process word).

- **Receive table** $R[0..N_R)$: channel (closed name word), pattern-program address, body (process word, code), environment address, persistent bit.

A receive in the soup is a *closure*: its body is shared code and its free variables are supplied by an environment, a heap-allocated vector of closed name words. Anything that becomes a *name* is fully instantiated instead (Section 5.8), since names must be canonical to be compared.

Lookup by channel is a comparison of one name word against all entries of the other table. In the first stages this is a register array with N_R comparators. It is replaced by a block-RAM hash table keyed on the channel word when table sizes outgrow that.

Top-level processes that are Boolean values are inert. They are counted in a two-counter *residue* register so that they remain observable.

5.5 Matcher and mask solver

The compiler turns each pattern into a *pattern program*, a short sequence of operations executed against a name word with an explicit cursor stack.

Operation	Effect
`NBIND`	bind the current name word to the next structural variable
`NEQ` i	fail unless the current name word equals structural variable i
`QZERO`	fail unless the current name word is `QZERO`
`QSEND`	fail unless the current name quotes a `SEND`; push the quoted payload, then continue with the channel
`BLEAF` T	raise `FAULT_SORT` unless the current name word is `QBOOL` b; set $mask \leftarrow mask \wedge (b\ ?\ T : \neg T)$
`END`	succeed if the cursor stack is empty

All Boolean leaves of one pattern are expressed by the compiler over a single ordered space of that pattern's mask variables, so their masks conjoin directly. The mask starts as all ones. The result of a successful run is the vector of structural bindings and the mask.

- The mask is the solution set: bit r is set exactly when the assignment $v_i \mapsto$ bit i of r solves every Boolean leaf.
- The pair is a redex if and only if the run succeeds and the mask is nonzero.
- The solver returns $count = \text{popcount}(mask)$ and, given $u < count$ from the arbiter, the index r of the u-th set bit. An occurrence of a mask variable outside any operator is a leaf whose table is a projection, and a non-linear Boolean pattern such as $v \,\&\&\, {\sim}v$ needs no special case. A variable that occurs under no operator is not a mask variable and is bound by `NBIND`, as in the AND gate of Appendix A.

The pure parts (mask update, popcount, select) are combinational functions and are among the Clash components of Section 6.

5.6 Arbiter and entropy

All nondeterminism of the calculus passes through one module with one operation: `choose(n)` returns $u \in [0, n)$ for $n \geq 1$. It is used three times per step: to pick a send, to pick a receive among those on the same channel, and to pick a solution row.

- **No pruning.** For every n and every $u < n$ there must be an entropy value for which `choose(n)` returns u. A priority encoder is forbidden: it would make the machine a refinement of the calculus and not bisimilar to it.

- **Entropy.** The reference source is a 32-bit maximal-length Galois LFSR with taps 32, 22, 2 and 1. A choice among n takes a 32-bit sample w and returns $\lfloor w\,n/2^{32} \rfloor$. The reference model advances the register 32 positions per choice, so that successive choices use disjoint bits. Hardware with a free-running register need not match this, because runs are compared through the decision log and not through seeds. A hardware random source may replace the register behind the same interface.
- **Fairness.** Only possibilistic fairness is required (**D14**). The three-stage choice does not induce the uniform distribution over redexes. When grading is added, each stage takes a weight vector in place of a count, and weighting a pair by its solution count makes the composite uniform; the `choose` interface is defined so that this is a change of argument type only.
- **Replay.** In replay mode the arbiter ignores entropy and takes its results from the decision log.

5.7 The step

The reactor is a state machine. States in which the abstraction function of Section 8 is defined are called *boundaries*.

1. **IDLE** (boundary). Drain deferred frees and accept ingress (Section 5.9). If the send table is empty, stay.
2. **PICK-SEND.** Choose a valid send s uniformly. Clear the per-scan *tried* vectors.
3. **PICK-RECV.** Compare the channel of s with every valid receive not yet tried for s. If none matches, mark s tried; if all sends are tried, the soup is *quiescent*: set the status bit and return to IDLE; otherwise choose another untried send. If some match, choose one, r.
4. **MATCH.** Run the pattern program of r on the quoted payload of s. On failure or zero mask, mark r tried for s and return to PICK-RECV.
5. **COMMIT.** Choose a solution row. Write the log record. Remove s; remove r unless persistent. Build the new environment: the environment of r extended by the structural bindings and, for each mask variable, the `QBOOL` of its bit in the row. Variables enter in a fixed order: structural variables in order of first occurrence in the pattern program, then mask variables in order of mask position, so that the last mask variable is the most recently bound.
6. **EXPLODE.** Walk the body of r with the new environment and add its top-level components to the soup (below).
7. Return to IDLE.

A failed scan changes no state, so the scan terminates after at most $N_S \times N_R$ matcher runs, and quiescence is detected exactly.

Explode. A work stack holds (process word, environment) pairs.

- `PAR`: push head and tail.
- `SEND`: resolve the channel through the environment, instantiate the payload (Section 5.8), insert into S, or emit on egress.
- `RECV`: resolve the channel, insert a closure into R.
- `DROP` of variable i: look up the name. `QZERO`: nothing. `QBOOL`: increment the residue. `QPTR`: push the named process with an empty environment. This is the general drop that the

replication encoding needs; because the quoted term is closed and shared, it costs no copying.

- `BFN`: evaluate the table at the environment values; increment the residue.
- `BOOL`: increment the residue. `ZERO`: nothing.

5.8 Instantiation

`inst(code, env)` returns the canonical closed term obtained by substituting the environment into the code. It is a post-order walk with an explicit stack that rebuilds nodes through the interner, tracking the number k of binders passed.

- A subterm whose free-variable depth is at most k is returned unchanged: it mentions no variable beyond the binders already passed. This is the common case and makes instantiation proportional to the open part of the code only.
- `VAR` i with $i < k$ is local and unchanged; with $i \geq k$ it is replaced by environment entry $i - k$, where entries are numbered from the most recently bound. Environment entries are closed, so no index shifting is needed.
- A `BFN` whose support becomes partly known is cofactored on the known variables and re-normalised; when fully known it becomes a `BOOL`.

5.9 Boundary

The environment is modelled as processes on designated channels, fixed in the image.

- **Ingress.** Each input port is a channel. A valid/ready interface inserts a send on that channel, accepted only in IDLE. In the first stages the payload is restricted to immediates, and the sort tag is checked against the port's declared sort; a mismatch is dropped and counted.
- **Egress.** Each output port is a channel on which the environment is the only receiver. A send on such a channel is emitted on the port's valid/ready interface and is not stored.

5.10 Decision log and faults

One record per step: step number, send slot, receive slot, solution row. A separate *insert* stream reports every entry added to S or R with its slot, and a *residue* stream reports residue increments. Together these let an external checker reconstruct the soup without reading the machine's internals.

Faults are sticky status bits that halt the reactor at the next boundary.

Fault	Raised when
`FAULT_SORT`	a Boolean operand or Boolean leaf meets a non-Boolean name. Unreachable for well-sorted programs (**D10**).
`FAULT_HEAP`	the heap, hash table or environment space is exhausted
`FAULT_STORE`	a send or receive table is full
`FAULT_STACK`	a work stack overflows

Resource faults are the only way the machine departs from the calculus, and they are detected, never silent.

Remark 2 (The sort flag is not a sort check). A Boolean law can erase an ill-sorted operand before anything inspects it: $*v$ `xor` $*v$ normalises to `false` and $*v$ && `true` to $*v$, whatever v is later bound to. `FAULT_SORT` therefore fires only for operands that survive normalisation. Its

never firing does not show that a program is well sorted, and Obligation 7 cannot be turned round. The sort checker carries the whole obligation.

5.11 Parameters

Parameter	Stage 1 value	Meaning
K	6	maximum Boolean support and pattern mask variables
N_S, N_R	64, 64	send and receive table sizes
A	28 (later)	heap address width; stage 1 has no dynamic heap
E	16	maximum environment length
D	32	work-stack depth
LFSR width	32	entropy state

6 Tooling and code organisation

6.1 Division of labour (D9)

Reference library (plain Haskell). Abstract syntax, normal forms, match sets, the one-step relation returning *all* redexes, and the node packing. This is the executable semantics. The compiler and the checker are built on it.

Clash. The pure, combinational datapath: truth-table operations (negate, combine over merged supports, minimise support, cofactor), mask update, popcount and select, the node hash, node encode and decode. These are ordinary Haskell functions over sized types, and they are written once and imported both by the reference library and by the Clash top entities, so there is a single source for each.

Bluespec (BH). Everything with state and scheduling: heap, interner, store, matcher sequencer, instantiator, arbiter, reactor, log, boundary. The calculus is a rewrite system and Bluespec's guarded atomic rules express each transition of Section 5.7 directly.

Clash-generated Verilog modules are imported into Bluespec as combinational foreign modules. If the two-tool flow proves costly, the fallback is Clash alone, with the reactor written as an explicit Mealy machine; nothing in this specification depends on Bluespec.

6.2 Rules for the Bluespec code

1. Every semantically visible choice goes through the arbiter module. The Bluespec scheduler resolves rule conflicts by a fixed urgency order, which is a priority encoder; it must never be the thing that decides which redex fires.
2. Any urgency annotation must carry a comment stating why the conflict it resolves is invisible at boundaries.
3. Each rule that changes the store or the heap states, as an assertion, the invariants of Section 8.4 it relies on.
4. No vendor primitives. Memories are inferred. One clock, synchronous reset.

6.3 Suggested module list

Module	Tool	Responsibility
`TruthTable`	Clash	table algebra and normalisation
`MaskSolve`	Clash	mask, popcount, select
`NodeCodec`	Clash	packing, hash, free-variable depth and sort bit
`mkHeap`	BH	node RAM, free list, reference counts
`mkInterner`	BH	hash-cons table, normalising constructors
`mkStore`	BH	send and receive tables, channel lookup
`mkMatcher`	BH	pattern-program sequencer
`mkInst`	BH	instantiate and explode walkers
`mkArbiter`	BH	`choose`, LFSR, replay input
`mkReactor`	BH	the step state machine
`mkLog`	BH	decision, insert and residue streams
`mkBoundary`	BH	ingress and egress ports

6.4 Flow

Source text → compiler → memory image. Bluespec and Clash → Verilog → simulation (Verilator) and synthesis (vendor or open flow). The same Verilog is the input to bounded formal checks.

7 Compiler and image

The compiler is software and is in the trusted base for sorts only (**D10**); every other property of its output is checked dynamically by the verification harness.

1. Parse; resolve variables to de Bruijn indices.
2. Run the sort checker; record each pattern variable's sort.
3. Normalise (Section 4); reject Boolean supports above K.
4. Compile patterns to pattern programs. Assign mask variables their positions in the pattern's mask space and extend each leaf table to that space.
5. Intern all code into a heap image using the same interner algorithm as the hardware, so that addresses in the image are canonical.
6. Explode the top-level process into initial send and receive tables.
7. Emit block-RAM initialisation files for the heap, hash table, reference counts, pattern store and the two tables, plus a symbol file mapping source names to name words for the test harness.

8 Verification

8.1 What "verified" means

Let α map a machine state at a boundary to a process:

$$\alpha(s) \;=\; \prod_{i \in S} c_i!(q_i) \;\mid\; \prod_{j \in R} \mathsf{inst}(recv_j, env_j) \;\mid\; \texttt{true}^{\,t} \mid \texttt{false}^{\,f}$$

where (t, f) is the residue and all terms are read back from the heap.

Obligation 3 (Soundness)**.** If the machine goes from boundary s to the next boundary s' by a step, then $\alpha(s) \to \alpha(s')$.

Obligation 4 (No pruning)**.** If $\alpha(s) \to Q$ and no resource fault occurs, there is a sequence of arbiter results for which the next boundary s' satisfies $\alpha(s') \equiv Q$.

Obligation 5 (Quiescence)**.** The quiescent bit is set at a boundary if and only if $\alpha(s)$ has no redex.

Obligation 6 (Replay)**.** Two runs from the same image with the same ingress sequence and the same decision log pass through identical boundary states.

Obligation 7 (Sort safety)**.** For a well-sorted image, `FAULT_SORT` is unreachable.

Obligations 3 and 4 together say that α is a bisimulation between the fault-free machine, observed at boundaries, and the calculus. Obligations 1 and 2 are the supporting facts about the language itself.

8.2 Layers of evidence

Layer	Method	Discharges
V1	Property tests of the reference normaliser against a brute-force decision of $\equiv$ on small terms; exhaustive for Boolean code with $k \leq 3$	Ob. 2 (reference)
V2	The Clash functions are the reference functions: test in GHC, then check generated Verilog against them by random co-simulation	datapath correctness
V3	Interner test: intern random raw terms through the machine's normalising constructors alone; addresses equal iff reference normal forms equal, and readback equals the reference normal form	Ob. 2 (hardware)
V4	Step-refinement checker (below) on random and directed programs	Ob. 3, 5
V5	Formal cover on the arbiter: for every $n \leq 64$ and $u < n$ a witness entropy value exists	Ob. 4 (arbiter)
V6	Redex coverage: on small programs, every reference redex at every visited state is taken under some seed	Ob. 4 (system)
V7	Replay test: record, replay, compare boundary traces	Ob. 6
V8	Assertions for the invariants below, in simulation and bounded model checking	supports all
V9	Bounded bisimulation check, in the reference, of persistent receive against the encoding; paper proof to follow	Ob. 1
V10	Instantiation commutes with normalisation, in the reference and in the machine, against textbook substitution on raw terms	Ob. 3 (instantiation)
V11	Mutation: each of a set of bugs injected into the machine and into the reference must fail some layer	adequacy of V1–V10

8.3 The step-refinement checker

The checker is a testbench component built on the reference library. It never reads machine internals other than the heap contents needed to decode addresses reported on the streams.

1. From the image and the insert stream it maintains a map from table slots to reference terms, and from the residue stream the residue counts. This is α computed incrementally.
2. On each decision record (send slot, receive slot, row) it checks that the triple is a redex of the current reference state: channels equivalent, the row is in the reference match set.
3. It computes the reference reduct, and at the next boundary checks that the inserts and residue increments since the record are, as a multiset and up to $\equiv$, exactly the top-level components of the reduct.
4. When the machine reports quiescence it checks that the reference state has no redex.

Any discrepancy is a failure with the offending step number, which the replay log reproduces exactly.

8.4 Invariants

I1. Every stored node is in normal form.

I2. No two distinct heap addresses hold equal nodes.

I3. Each reference count equals the number of references from nodes, tables, environments and stacks.

I4. Every channel word and every payload word in the store is closed.

I5. A `DROP` node's name is a `VAR`; a `BFN` has minimal, ascending support, is not constant and is not a projection.

I6. A persistent receive is never removed by a step.

I7. At a boundary all work stacks are empty.

I8. The mask is nonzero at COMMIT, and the chosen row is a set bit.

I9. Exactly one decision record is written per step.

8.5 Directed test programs

Program	Checks
Forwarder, AND, XOR gates	forward evaluation; one solution per match
Half adder, full adder	composition of gates through channels
Inverse XOR (Appendix A)	two solutions, both reached; empirical frequencies
Inverse AND on `false`	three solutions, all reached
Unsatisfiable pattern v && $\sim v$ on `true`	no redex; send remains; quiescence
Two receivers, one send	COMM race; both outcomes reached
Replication by encoding	general drop; an unbounded run. Hash-consing shares every copy, so the heap does not grow: the run ends on the receive-table bound when the replicated process leaves receives behind, and otherwise never
Persistent receive	same traces as the encoding, fewer steps
Equal names, different syntax	@(`true` && `false`) and @`false` as channels are the same channel
Ill-sorted image (negative test)	`FAULT_SORT` is raised and halts

9 Staging

Stage	Content	Exit criterion
1. Boolean core	Names are immediates or image constants; shallow patterns; code in a ROM, no dynamic heap; store, matcher, solver, arbiter, reactor, log	V2, V4–V8 pass on gate and inverse-gate programs, on an FPGA and in simulation
2. Reflection	Heap, interner, reference counts, instantiation, general drop	V1, V3 pass; replication by encoding runs
3. Full patterns	`QSEND` and deeper patterns; persistent receive; ingress of quoted terms	V9 passes; full directed suite
4. Typed fast path	Channels typed Boolean-only are elaborated to one-bit links and gates; the tuple-space machine remains the fallback	elaborated and interpreted runs agree under the checker
5. Parallel reduction	Several reactors on disjoint channels	Ob. 3, 4 restated for interleavings

Later attachments, not staged here: cost accounting as a per-step debit at COMMIT; graded choice through the arbiter's weight interface; the combinator instruction set as an alternative code representation in the same heap.

10 Open question and recorded limits

Open: matching under a binder. The pattern $\texttt{for}(v \leftarrow p)\, d$ matches a receive and its body. The body of the matched receive has a free bound variable, so a variable of d cannot simply bind "the name of the body". The candidate readings are: bind the body as an abstraction; or allow d to mention v and match up to renaming. The machine supports neither until this is settled, and the pattern is rejected by the compiler in the meantime.

Recorded limits. These follow from confirmed decisions and are listed so that an implementer does not mistake them for oversights.

- **Board.** None is fixed (**D16**). Block-RAM budgets are set per board through the parameters.
- **Trust.** The compiler is trusted for sorts (**D10**). A self-defending chip would need a hardware loader and a type system checkable in time linear in the image.
- **Support bound.** $K = 6$ (**D12**). Lifting it needs reduced ordered decision diagrams in the interner.
- **Fairness.** Possibilistic only (**D14**). A prescribed distribution over redexes enters through the arbiter's weight interface.
- **Top-level Booleans.** Kept in the residue counters.
- **Operator patterns.** Closed Boolean values only (**D13**).

11 Status and changes in draft 0.3

Status. A reference implementation, `rhoref` 0.1.0, now exists: the executable semantics, a word-level model of the machine of Section 5, and the step-refinement checker, in plain Haskell.

Its suite discharges layers V1, V3 to V7, V10 and V11 in software, and V9 at the level of egress traces. Layers V2 and V8 concern generated Verilog and wait for the hardware. Nothing has been proved; Obligation 1 in particular is tested and not yet proved.

Corrections. Each was found by building or testing the reference, which implements the corrected reading.

1. **Free-variable depth** (Sections 5.2, 5.8). The node header carries a free-variable depth in place of a closed bit, which cannot be computed for a receive from its children.
2. **The sort flag is incomplete** (Section 5.10, **D10**). Boolean laws can erase an ill-sorted operand, so `FAULT_SORT` is a debugging aid and not a sort check.
3. **Mask variables** (Sections 3.4, 5.5, 5.7). Draft 0.2 said a bare Boolean-sorted variable is a projection leaf, while its appendix bound one with `NBIND`. The rule is now stated once: a variable is a mask variable exactly when it occurs under a Boolean operator in the pattern. The order in which variables enter the environment is fixed.
4. **A Boolean leaf on a non-Boolean** (Sections 3.4, 5.5) is a sort fault, not a failed match.
5. **Replication by encoding** (Section 8) ends on the receive-table bound, or never; it does not exhaust the heap.
6. **Entropy sampling** (Section 5.6) is specified: the LFSR polynomial, the map from a sample to a choice, and the reference model's 32 positions per choice.

The verification plan gains two layers. V10 was added because instantiation is where normal forms are re-established at run time. V11 was added because two injected bugs survived the first suite, which showed that the interner had been tested only on terms the reference had already normalised; V3 is restated accordingly.

A Worked examples

Let x, y, z, l, r be distinct closed names.

An AND gate

$$\texttt{for}(a \leftarrow x)\,\texttt{for}(b \leftarrow y)\,z!(*a \mathbin{\&\&} *b) \;\mid\; x!(\texttt{true}) \;\mid\; y!(\texttt{false})$$

- **Code.** Inside the inner body, b has index 0 and a has index 1. The payload is a `BFN` with support $(0, 1)$ and table $T = 1000_2$: only row $r = 3$, where both are true, is set.
- **Initial soup.** One receive on x with pattern program `NBIND; END` and an empty environment; sends $(x, \texttt{BOOL}\ 1)$ and $(y, \texttt{BOOL}\ 0)$.
- **Step 1.** The send on y has no receive, so the only redex is on x. The mask is all ones over zero Boolean variables, one solution. The environment becomes [@`true`]. Explode inserts a receive closure on y with that environment.
- **Step 2.** COMM on y. The environment becomes [@`true`, @`false`], so index 0 (b) is false and index 1 (a) is true: row $r = 2$. Instantiating the payload reads bit 2 of T, which is 0. Explode inserts the send $(z, \texttt{BOOL}\ 0)$.
- **Result.** $z!(\texttt{false})$, and the machine is quiescent.

An inverse XOR

$$\texttt{for}(@(p\ \texttt{xor}\ q) \leftarrow x)\ (l!(*p) \mid r!(*q))\ \mid\ x!(\texttt{true})$$

- **Pattern program.** `BLEAF` T; `END`, with p at mask position 0, q at position 1 and $T = 0110_2$.
- **Match.** The payload is `QBOOL` 1, so the mask is $T = 0110_2$: rows 1 and 2, count 2.
- **Choice.** The arbiter returns $u \in \{0, 1\}$. With $u = 0$ the row is 1: $p = \texttt{true}$, $q = \texttt{false}$. With $u = 1$ the row is 2: $p = \texttt{false}$, $q = \texttt{true}$.
- **Result.** Either $l!(\texttt{true}) \mid r!(\texttt{false})$ or $l!(\texttt{false}) \mid r!(\texttt{true})$. Both must be reachable, and the row is recorded in the log. Had the payload been `false`, the mask would be $\neg T = 1001_2$ and the two outcomes would be the equal pairs.

The second example is a one-bit entropy tap written in the calculus itself: the quality of the machine's random source is observable by a program.